



**Capstone Project – Python Program
Logic and Functions
Practical Skills Module 3 Capstone Project**

[Visit our website](#)

Capstone Overview

This capstone project forms part of your **Practical Skills Module 3** and is designed to assess your ability to develop structured Python programs using control flow, operators, functions, modules, and input and output.

In this capstone, you will complete four practical tasks that integrate key programming concepts, including decision-making, loops, operators, functions, date and time handling, and user interaction. You will also work with external modules and file handling to create more interactive and functional programs.

Each task is based on a real-world scenario and requires you to apply multiple concepts together. You will be expected to write structured, well-organized Python programs that demonstrate clear logic, correct use of functions, and accurate outputs.

By the end of this capstone, you will have developed a set of Python programs that demonstrate your ability to build interactive and logically structured applications.

You will show that you can:

- Use decision structures and loops to control program flow
- Apply operators to perform calculations and comparisons
- Define and use functions with parameters and return values
- Use built-in and advanced Python functions effectively
- Work with date, time, and calendar functionalities
- Import and use Python modules
- Accept user input and produce meaningful output
- Write structured programs that solve practical problems

Practical Module Scope

This capstone integrates the following practical skills:

- **PM-03-PS01:** Create decisions in Python
- **PM-03-PS02:** Use operators
- **PM-03-PS03:** Define functions in Python
- **PM-03-PS04:** Declare date, time, and calendar

- **PM-03-PS05:** Use import, input, and output functionalities

Capstone Learning Outcomes

By completing this capstone, you will be able to:

- Use if, if-else, and if-elif-else statements to control program logic
- Create and manage for loops and while loops
- Control loop execution using break, continue, and pass
- Perform arithmetic, comparison, logical, and bitwise operations
- Define functions using parameters and return values
- Apply built-in Python functions and tools effectively
- Manage variable scope using local, global, and nonlocal variables
- Import and use Python modules in a program
- Work with date and time objects and format outputs
- Accept user input and process it correctly
- Produce structured and user-friendly output

Instructions

You are required to complete all four practical tasks in this capstone. Each task must be completed in a separate Python file using the specified filename.

Read each scenario carefully before you begin. Make sure you understand what the program is expected to do and which Python concepts you need to demonstrate.

When completing this capstone:

- Create one Python file for each practical task and use the exact filename provided
- Write code that is clear, well-structured, and easy to read
- Use meaningful variable names, appropriate indentation, and consistent formatting
- Ensure that each program includes all the required functionality listed in the task

- Display output clearly so that the purpose and result of each step can be easily understood
- Test each program before submitting it to make sure it runs without errors
- Apply the correct Python concepts, functions, operators, modules, and control structures as required in each task



Practical tasks

Practical Task 1: Inventory Control System

A warehouse manager wants to automate inventory tracking and decision-making. The system must monitor stock levels, classify inventory, and simulate restocking processes. Create a Python file named **inventory_control.py**.

Create a Python program that:

- Stores a set of items and their quantities (for example: {"Item A": 10, "Item B": 20, "Item C": 5})
- Uses an if statement to check if a specific item has low stock (for example, less than 10 units)
- Uses an if-else statement to display whether stock is sufficient or needs restocking
- Extends this logic using if-elif-else to classify stock as:
 - low stock (less than 10)
 - sufficient stock (10 to 30)
 - overstocked (above 30)
- Uses a for loop to iterate through all items and display their stock status
 - example output: Item A: Low stock
- Uses a while loop to simulate restocking an item until it reaches a target level (for example, 30 units)

- Uses `break` to stop the loop if a maximum level (for example, 50 units) is exceeded
- Users continue to skip items that already meet the required stock level
- Defines a function such as `send_alert(item_name)` and uses the `pass` statement as a placeholder
- Ensures all outputs are clearly labeled and easy to read

Practical Task 2: Student Performance Analyzer

A school requires a system to evaluate student performance. The program must calculate results, assign grades, and determine eligibility for awards. Create a Python file named **`student_performance_analyzer.py`**.

Create a Python program that:

- Stores marks for three subjects (for example: Math = 85, Science = 90, English = 78)
- Calculates:
 - total marks
 - average marks
 - percentage
- Uses if-elif-else statements to assign a grade based on the percentage:
 - Grade A: 90 or above
 - Grade B: 75 to 89
 - Grade C: below 75
- Uses logical operators to determine award eligibility
 - example: eligible if percentage is 90 or above
- Checks whether a student name exists in a list such as: `["Alice", "Bob", "Charlie"]`
- Uses the `is` operator to compare two variables
 - example: `grade1 = "A"` and `grade2 = "A"`

- Performs a weighted calculation using different subject weights
 - example: Math = 40%, Science = 35%, English = 25%
- Uses at least one bitwise operator to demonstrate comparison between values
- Prints all results clearly with labels
 - example: Final Grade: B

Practical Task 3: Order Processing System

An e-commerce company requires a system to process customer orders, apply discounts, and generate simple reports using functions and built-in tools. Create a Python file named **order_processing_system.py**.

Create a Python program that:

- Defines a `main()` function and calls it to run the program
- Creates a function such as:
`calculate_discount(price, discount_rate)`
 which returns the discounted value
- Creates another function such as:
`process_order(order_id, status)`
 which returns the updated order status
- Uses a lambda function for a simple calculation
 - example: adding tax to a price
- Uses built-in functions such as:
 - `abs()` to calculate differences
 - `round()` to format values
 - `range()` to iterate through order IDs
 - `map()` to apply a function to multiple values
- Uses the Counter class to count items in a list
 - example: product categories
- Uses `enumerate()` to loop through data with index values
- Uses `time.sleep()` to simulate a short delay during processing

- Checks variable types using `type()` and `isinstance()`
- Demonstrates variable scope:
 - a local variable inside a function
 - a global variable updated using the `global` keyword
 - a `nonlocal` variable inside a nested function
- Prints structured output showing the results of each operation

Practical Task 4: Event Scheduler and Quiz System

You are developing an interactive system that combines event scheduling and a quiz application. The program must manage dates and times, interact with users, and store results. Create a Python file named **event_and_quiz_system.py**.

Create a Python program that:

- Imports modules such as:
 - `datetime`
 - `time`
 - `random`
 - `calendar`
 - `sys`
- Displays the current date and time using:
 - `datetime.now()`
- Displays only the current date using:
 - `date.today()`
- Extracts and prints components such as:
 - year, month, day, hour, and minute
- Formats the date using `.strftime()`
 - example: Friday, 13 May 2025
- Converts a string into a date using `.strptime()`

- example: "2025-06-30 15:00"
- Uses `time.sleep()` to simulate a delay or countdown
- Displays a calendar for a selected month using the calendar module
- Asks the user quiz questions using `input()`
- Checks answers and keeps track of a score
- Displays feedback such as:
 - Correct or Incorrect
- Records results (name, score, time taken) in a text file
 - example file: `quiz_results.txt`
- Allows the user to exit the program using a command such as "exit" and `sys.exit()`
- Ensures all interaction is clear and user-friendly

Submission Requirements

You must submit the following Python files:

- `inventory_control.py`
- `student_performance_analyzer.py`
- `order_processing_system.py`
- `event_and_quiz_system.py`



Take note

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.
